

(Blind) Multimodal Bot Detection on X

Hamza Rashid
McGill University
COMP 400

Abstract—This research investigates multimodal bot detection on X, in a setting where labels are not known a priori, and graph data is not available. Our work begins with benchmarking conventional classifiers—a random forest on text, temporal, and topic-distribution features, yielding moderate baseline performance. Noticing that bots copy tweets and descriptions from genuine users, we explore various multimodal approaches to combat manipulated features, with an emphasis on DNA-inspired behaviour modeling to encode user activity more dynamically. For this, we embed the user description with TwHIN-BERT and model posting behavior with two DNA formulations, one that captures content across posts, and another capturing inter-post times. We find that CNN embeddings of inter-post time DNA may capture latent patterns in temporal behaviour that conventional methods miss. We investigate three methods for processing the modalities and final fusion; a GMU (gated multimodal unit), a cross-attention mechanism, and a sparsely gated MoE (mixture-of-experts) with a self-attention fusion layer and a convolution learned on attention weights. We find that the MoE is not only our best method, but is also the smallest of our multimodal models, and does not require any treatment of the class imbalance found in bot detection datasets. Our findings show that a multimodal approach is robust against sophisticated bots, having manipulated features in at least one modality, and that the divide and conquer paradigm helps address the diversity of user communities on X.

I. INTRODUCTION

Social media platforms like X (formerly Twitter) have enabled unprecedented global connectivity. As an open platform is vulnerable, X is fertile ground for malicious content, often originating from automated accounts—bots—that manipulate discourse, spread misinformation, or artificially amplify content. Detecting and mitigating such bot activities remains a challenging yet essential task for preserving authentic online interactions.

Traditional approaches to bot detection typically rely on supervised classification methods trained on manually labeled datasets. Yet, in real-world scenarios, labeled data are often scarce, outdated, or non-representative of the evolving behaviors exhibited by sophisticated bots. Consequently, there is a pressing need for detection methods that generalize beyond known bot patterns, especially in contexts where labeled data is not readily available.

In this report, we address this challenge by evaluating a range of bot detection strategies, beginning with classical machine-learning techniques enhanced by extensive feature engineering. We initially employ a random forest classifier incorporating textual features, temporal posting patterns, and inferred topic distributions, achieving moderate baseline performance. Our analysis reveals that traditional temporal features often fail to effectively distinguish sophisticated bots

from genuine users, underscoring the limitations of classical approaches.

Building on these insights, we introduce deep multimodal architectures designed to exploit richer behavioral patterns. Inspired by the concept of digital DNA, our method integrates user descriptions embedded via TwHIN-BERT and models posting behavior through two distinct DNA formulations: one capturing the content across posts and another encoding inter-post timing patterns. In addition to the known efficacy of content DNA (Ilias et al., 2023), our experiments demonstrate that CNNs applied to temporal-DNA-generated images effectively uncover nuanced behavioral patterns overlooked by conventional methods. As most of the frontier literature overlooks temporal behaviour, this result signifies that it could be worth further investigation.

We further explore several fusion techniques to integrate multimodal signals, including gated multimodal units (GMUs), cross-attention mechanisms, and a novel application of a sparsely gated mixture-of-experts (MoE) (Shazeer et al., 2017) architecture. Our results highlight the MoE model’s superior performance and efficiency, notably without necessitating explicit strategies to address class imbalance. Overall, our research emphasizes the robustness of multimodal approaches and illustrates the potential of MoE architectures to adapt effectively to the diverse user communities found on X.

II. DATASET AND EXPERIMENTAL SETUP

Our experiments were conducted within the “Bot Or Not” (B.O.N.) competitive infrastructure, designed specifically for assessing bot detection strategies in realistic social media scenarios. This environment featured interactions between bot and detector teams, simulating genuine activities on the X platform. The primary goal was to evaluate the effectiveness of detection methodologies under conditions closely resembling live social media dynamics.

A. Data Collection and Characteristics

The dataset comprised real posts gathered from the X API across multiple collection sessions, each lasting the same duration but occurring on different days. For each session, posts were initially collected based on predefined sets of topical keywords relevant to that session. After gathering initial posts, users flagged as obvious bots were removed, and the dataset was expanded by including all additional posts from the remaining users within the session period.

The preliminary data collection process can be summarized as follows:

- Text-only Posts: Posts containing images, videos, or memes were excluded.
- Static Metadata: Dynamic metadata such as likes, comments, and follower counts were excluded to maintain consistency and simplicity.
- Anonymized Links and Mentions: Hyperlinks were standardized to “https://t.co/twitter_link”, and user mentions were anonymized to “@mention” to prevent easy verification of authenticity.
- Activity Thresholds: Each included user produced between 10 and 100 posts during the session period, ensuring sufficient per-user data.

Datasets were structured into two main JSON formats:

- Posts Dataset: Included text content, timestamp, unique post ID, author ID, and language.
- Users Dataset: Contained user-specific metadata, including user ID, username, display name, profile description, location, tweet count, and a calculated activity-based z-score.

B. Session Structure

Sessions were divided into multiple sequential sub-sessions to mimic the real-time flow of social media feeds, gradually introducing new posts to participants. Bot teams generated content incrementally across these sub-sessions, with the requirement to post between 10 and 100 posts per bot by session completion. Detector teams subsequently analyzed the fully compiled datasets post-session to identify bots, providing both binary verdicts and confidence scores for each account.

C. Technical Setup

Participant submissions (bot and detector code) were managed via public GitHub repositories cloned into Docker images deployed on EC2 instances, ensuring standardized computing resources and fair evaluation conditions. Submissions were required to execute fully within a one-hour timeframe, after which automated shutdown procedures terminated any ongoing processes.

III. METHODOLOGY

Our approach involved a systematic progression from classical machine learning methods to multimodal neural-based architectures, aimed at overcoming limitations observed at each stage. While the lack of graph and user meta data limited the scope of our research, we explored methods capable of incorporating such features. Our primary goal was to model user behaviour from a variety of perspectives since nontrivial bots manipulate features in at least one aspect (e.g., tweet content, posting times, or interactions such as retweets, likes, and follower and following relationships).

A. Baseline - Random Forest

Our baseline method involved training a Random Forest classifier using various handcrafted and learned features from user posts. We organized our experiments across multiple data collection sessions to assess how well feature-based

models generalize across batches of social media activity. The sub-approaches below correspond to progressively enriched feature sets tested on different sessions. We employ a random forest for its flexibility with data types (allowing rapid feature exploration), and proven efficacy in bot detection, most notably in Botometer (Davis et al., 2016).

1) Session 11 – Embeddings + LDA: For session 11, we extracted tweet embeddings for each user with all-MiniLM-L6-v2 (Reimers and Gurevych, 2021), and applied LDA (Blei et al., 2003) topic modeling under the assumption that bots target specific topics, resulting in sparse vectors with relatively large values in a few entries. Aggregating posts into a single document simplified the problem but at the cost of granularity, which we later addressed. We performed minor text preprocessing, such as stopword removal, and converting urls, mentions, and hashtags into special tokens (e.g., <URL>) in order to remove noise from the topic modeling process. Finally, a Random Forest with 100 trees and class weights inversely proportional to occurrences was trained and evaluated using stratified 80-20 train-test splits.

Result: The model exhibited low confidence and marked all samples as negatives. This likely owes to its reliance on predefined topics from training data, which vary across sessions, resulting in sparse vectors on unseen data.

Limitations:

- Did not use individual tweets as independent samples, thereby lacking granularity.
- Static topic modeling limited generalization to new sessions.
- Removing stopwords likely degraded the quality of the contextual text embeddings.

2) Session 12 – Mean Embedding + BERTopic + Temporal Statistics: In this session, we shifted to using per-tweet embeddings and adopted BERTopic (Grootendorst, 2022) for more dynamic topic modeling. Additional temporal statistics were introduced to capture behavioral rhythms. As an extra measure for addressing the low confidence scores, we performed min-max scaling of the confidences after inference.

Implementation Summary:

- Tweet-Level Processing: Individual tweets were embedded and clustered into topics using BERTopic with HDBSCAN (McInnes et al., 2017).
- Aggregation: User-level features were computed via:
 - Mean of tweet embeddings.
 - Normalized topic distribution.
- (Naive) Temporal features: Mean and standard deviation of raw timestamps.
- Static features: Post count.

Result: The model achieved moderate results in this session, marking positives and having higher confidence scores overall. This result likely owes to (1) scaled confidence scores, (2) time based features providing obvious signals and (3) batch-agnostic topic modelling providing informative features to the classifier. The time-based features likely provided the

most utility, since many bots at this point did not modify their posting times.

Limitations:

- Topic feature vectors were padded to a fixed length, potentially introducing sparsity and noise.
- Temporal features were still limited to coarse statistics.

3) *Session 13 – Enhanced Temporal Behavior Modeling:*

Session 13 introduced a novel temporal feature: duplicate post timestamps within bucketed time intervals, designed to capture artificial timing artifacts used by bots.

Implementation Summary:

- Building on Session 12, we added:
- Inter-post Interval Statistics: Mean and standard deviation of time gaps between posts.
- Bucketed Duplicate Counts: Timestamps were partitioned into 10 intra- and inter-chronologically-ordered buckets, and duplicate timestamps were computed per bucket.
- Features Used: Mean-pooled tweet embeddings, topic distributions, post frequency, inter-post intervals, and bucketed duplicates.

Result: This was the strongest-performing Random Forest baseline. The bucketed duplicates provided a clear signal in identifying bots with repeated behavior patterns or scheduled posts.

Limitations:

- Still relied heavily on session-specific BERTopic clusters.
- Lacked explicit modeling of inter-feature dependencies (e.g., topic-time correlations).

4) *Session 14– Topic Statistics + Temporal Entropy + Intra-Topic Similarity:* This final Random Forest setup expanded feature engineering to include statistical, temporal, and semantic regularities observed in bot behavior. The design aimed to encode not just what a user posts, but also how, and when, consistently.

Implementation Summary:

- Refined Preprocessing: Hashtags were partially preserved (`#arena` $\rightarrow$ `<HashtagMention:arena>`), preserving topic cues often used by bots.
- Normalized Embeddings: Sentence embeddings (MiniLM-L6-v2) were explicitly normalized to prevent variance from dominating downstream signals.
- Topic Modeling: BERTopic + HDBSCAN, followed by:
 - Topic Histogram (topic mention counts across topics)
 - Summary Statistics: Mean, variance, standard deviation, entropy, and dominance ratio across topic indices.
- Temporal Signals:
 - Mean, standard deviation, and variance of time between posts.
 - Bucketed duplicate timestamps (tolerance-aware) to flag spam bursts.
- Intra-topic Semantic Consistency:
 - Mean, median, and standard deviation of pairwise cosine similarities across tweets within each user-topic group.

Result: In session 14, the bots became more difficult to distinguish. That is, this model outperformed its predecessor under the same training conditions when applied to the previous session, but performed worse on the latest. Despite the drop in performance, the model achieved nontrivial results, second only to the previous session. A core limitation was the increased feature dimensionality, introducing noise to the model and potentially stressing interpretability.

B. *Summary and Reflection on Random Forest Baseline*

Our progression through four sessions using Random Forest classifiers illustrates the evolving challenge of distinguishing bots from humans on X through feature engineering alone.

What Went Right:

- Rich Feature Design: Session 14 demonstrates that carefully designed user-level features (temporal, semantic, topical) can yield strong performance using classical models despite evolving bots.
- Dynamic Topic Modeling: Transitioning from static LDA to BERTopic+HDBSCAN enabled models to better capture cross-session topical drift.
- Behavioral Signals: Features such as inter-post intervals and intra-topic semantic consistency offered important signals for detecting scripted or bursty behavior patterns.

What Went Wrong:

- Sensitivity to Session Artifacts: A non-pretrained BERTopic model was applied to each session, which might have created inconsistencies in how topic-features were interpreted by the classifier.
- Appropriateness of features: Despite being a universal approximator, Random Forest’s are typically geared towards categorical features; introducing several related continuous features like embeddings can introduce noise. To utilize such features effectively, it would be more appropriate to employ an MLP (multi-layered perceptron).
- Scalability Concerns: Computing transformer embeddings, pairwise similarities, multiple topic statistics, and padded histograms scales poorly for large user datasets.

While the Random Forest baseline allowed us to test many ideas rapidly, its limitations became evident in detecting sophisticated or adaptive bots (increasing complexity of feature engineering was needed to keep up). We also found that many bots copied tweets and user descriptions from genuine users, thereby reducing informative signals in our features. Further, while we were able to encode user behaviour from various perspectives, our model relied on many static features (e.g., mean and standard deviation of time between posts), which are also prone manipulation. Thus, the observed manipulation and diminishing returns on feature engineering motivated the shift to multimodal methods whose underlying features and architecture can adapt to evolving behaviours on X.

C. *Multimodal Architectures*

Recognizing the constraints of feature engineering, we explored multimodal neural architectures inspired by recent

literature “Multimodal Detection of Bots on X (Twitter) using Transformers” (Ilias et al., 2023), and “BotMoE: Twitter Bot Detection with Community-Aware Mixtures of Modal-Specific Experts” (Liu et al., 2023). Our modalities include transformer-based description and tweet embeddings alongside digital DNA representations (content and temporal).

Evaluated architectures include:

- Gated Multimodal Unit (GMU): Dynamically weighting modalities.
- Cross-Attention Fusion: Learns cross-modal interactions through multihead attention.
- Mixture-of-Experts (MoE): Subsets of the models specialize in different user types, with attention-based fusion and a learned convolution over attention weights.

These architectures not only provide greater modeling capacity, but also offer more robustness against feature manipulation, as they do not rely on manually crafted statistics. Instead, they operate directly over sequences of observed user behavior and learn to extract structure.

1) **DNA Embedding Extraction:** To model user behavior beyond text embeddings, we construct symbolic sequences known as digital DNA, across the user’s timeline chronologically, representing content, timing, and emoji usage across tweets. These sequences are then converted into images and embedded via a CNN encoder. In all cases, the resulting character sequence is transformed into an image as follows:

- 1) Each symbol is mapped to an 8-bit grayscale intensity value (e.g., N $\mapsto$ 0, U $\mapsto$ 64, ..., X $\mapsto$ 255).
- 2) The sequence of grayscale values is reshaped into a square matrix: if the sequence has length L , the smallest $n \in \mathbb{N}$ such that $n^2 \geq L$ is selected. The sequence is zero-padded to length n^2 and reshaped to $(n \times n)$.
- 3) This grayscale image is resized depending on the CNN encoder, using nearest-neighbor interpolation.
- 4) The image is then converted to RGB format by replicating the grayscale channel, and the resulting tensor has shape: $3 \times \text{image_width} \times \text{image_height}$.
- 5) The tensor is normalized using ImageNet statistics and passed through a pretrained CNN encoder.
- 6) The output features are either (1) average pooled and projected into a fixed embedding space, or (2) directly extracted from the last hidden layer of the CNN encoder.

Content DNA. Each tweet is mapped to a symbol based on the type of entities it contains:

- N — No entities (plain text)
- U — Contains a URL
- H — Contains one or more hashtags
- M — Contains one or more mentions
- X — Contains a combination of multiple entity types

Time DNA. Time DNA encodes inter-post intervals by assigning symbols based on the time elapsed between consecutive tweets:

- n — First post (no delta)
- o — < 1 second
- s — ≥ 1 second and < 1 minute

- m — ≥ 1 minute and < 1 hour
- h — ≥ 1 hour and < 6 hours
- u — ≥ 6 hours and < 12 hours
- x — ≥ 12 hours

Emoji DNA. We categorize emojis in tweets using a precompiled lookup table. Each tweet is assigned a category based on its emojis:

- S — Smileys & Emotion
- P — People & Body
- A — Animals & Nature
- F — Food & Drink
- T — Travel & Places
- R — Activities
- O — Objects
- Y — Symbols
- L — Flags
- N — No emoji present
- X — Multiple emoji categories

In all cases, these DNA images serve as behavioral *fingerprints*, allowing CNNs to learn latent signals over them.

2) **Gated Multimodal Unit (GMU):** The GMU allows the network to dynamically control how much each modality contributes to the final hidden representation. The output is used (with possibly further transformation) downstream for classification.

In the bimodal setting, where we consider two input modalities $\mathbf{x}_v$ and $\mathbf{x}_t$, the equations governing the GMU are as follows:

$$\mathbf{h}_v = \tanh(\mathbf{W}_v \cdot \mathbf{x}_v) \quad (1)$$

$$\mathbf{h}_t = \tanh(\mathbf{W}_t \cdot \mathbf{x}_t) \quad (2)$$

$$\mathbf{z} = \sigma(\mathbf{W}_z \cdot [\mathbf{x}_v, \mathbf{x}_t]) \quad (3)$$

$$\mathbf{h} = \mathbf{z} * \mathbf{h}_v + (1 - \mathbf{z}) * \mathbf{h}_t \quad (4)$$

$$\Theta = \{\mathbf{W}_v, \mathbf{W}_t, \mathbf{W}_z\} \quad (5)$$

Here, $\mathbf{h}_v$ and $\mathbf{h}_t$ are hidden states learned on modalities $\mathbf{x}_v$ and $\mathbf{x}_t$, $[\cdot, \cdot]$ denotes concatenation, and Θ denotes learnable parameters. Cross-modal interactions are learned in the parameters $\mathbf{W}_z$, the weighting mechanism $\mathbf{z}$ then applies a sigmoid activation σ on the projected modalities to determine how much each one contributes to the final fused representation. In practice, implementations tend to depart¹ from some of the original equations, including ours.

Resembling the work of Ilias et al. (2023), our initial efforts with the GMU used the following modalities:

- Text: User description embedding from TwHIN-BERT.
- DNA: Content-DNA embedding from MobileNetV2.

The first iteration consisted of a two layer classification head applied to the GMU’s output. This model marked all samples as negatives, and several adjustments were needed. Subsequent iterations introduced refinements such as: modality-specific

¹Arevalo et al. (2017)’s own demo <https://github.com/johnarevalo/gmu-mmimdb/blob/master/model.py>; omits separate projections for each modality and applies the nonlinearity directly to the inputs.

projections, layer normalization, global residual connections, and modality-specific dropout.

a) **Incorporating Time-DNA:** While refining our GMUs, we incorporated time-DNA embeddings as a third sub-modality. This addition improved performance, supporting our intuition that a dynamic temporal feature can help capture richer distinctions between bots and humans.

While our GMU iterations demonstrated nontrivial improvements, development required numerous adjustments to stabilize training and optimize performance. This raised concerns about generalizability; can such a carefully balanced architecture handle different modalities or scale to more modalities?

Given these concerns, we transitioned to a cross-modal attention approach, expecting that it could achieve nontrivial performance with fewer architectural tweaks.

3) **Cross-Attention:** We first learn an intermediate representation of the DNA embeddings (content and time) to later apply cross-attention with the user description. We found that employing a GMU to learn a fused DNA representation was effective, despite its intended multimodal use.

Our cross-attention method (illustrated in Figure 1) fuses text (user description) and DNA in the following way:

- Apply a simple GMU to the DNA embeddings to create an intermediate representation of fused-DNA.
- Apply cross-attention layers to the description and fused-DNA in both directions.

To perform cross-modal attention, we employ two multihead-attention layers, computing scaled dot-product attention (Vaswani et al., 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (6)$$

in one direction where the description modality is the query (Q) and fused-DNA provides the key (K) and value (V), and another direction in which the modalities swap roles. Here, d_k denotes the dimensionality of the query and key. A mean pooling is then applied to attention outputs to form a final fused representation.

With the fused representation, we began with a two layer classification head and iteratively improved our architecture. This required less refinements than our GMU approach.

a) **Sessions 16-17 - Description, Content, Time:** For sessions 16 and 17, we employed the cross-attention approach, and trained the model with weighted cross-entropy loss to deal with the class imbalance. The model’s design follows a structured flow employing conventional methods such as residual connections and layer normalization:

- Pre-Layer normalization (Xiong et al., 2020) and residual connections are applied at each attention stage.
- The cross-attention outputs are concatenated into a sequence and fed through a single position-wise FFN for better regularization.
- Pre-layer normalization is applied to the tokens and a residual connection is added after the FFN output.

- A mean pooling is applied to the FFN outputs, and a two-layer classification head generates the final logits for bot detection.

The model underperformed relative to some of our random forest attempts. However, the bots had improved significantly by session 16, and a drop in performance was expected given the model’s complexity and the limited fine-tuning conducted at this stage. The model showed strong cross-lingual transfer, achieving the highest recall in french (session 17) despite training only on english data, demonstrating the importance of a multimodal approach.

b) **Sessions 18-19 - Description, Content, Time, Emoji:**

For sessions 18 and 19, the model’s GMU was extended to fuse three DNA embeddings: content, time, and emoji. Our intuition was that LLM-powered bots exhibit obvious tells in their use of emojis across posts. Further, it was trained with focal loss (Lin et al., 2017), which was more effective than weighted cross-entropy for dealing with the class imbalance.

Bot recall did not improve, but since we had more time to fine-tune the model, we achieved a higher precision and f1 score than our previous attempt. The sparse and high-dimensional nature of the emoji embeddings may have introduced noise to the model, which would explain the increased sensitivity to hyperparameter configurations. We therefore removed the emoji-DNA from subsequent approaches.

This experiment revealed the challenges of integrating sparse representations like emojis. Future refinements may involve curating a more appropriate categorization system for emojis.

Although a cross-attention approach can be effective, it is computationally expensive to scale beyond two modalities. For example, three modalities would require six multihead-attention layers to consider each cross-modal interaction in both directions.

A natural alternative is to employ a single self-attention block where each token corresponds to a modality. A key challenge in this approach is learning modality representations to provide to the self-attention layer. We find that MoEs learn these representations effectively, and in particular, are better at modelling the diverse user behaviours found on X.

4) **Mixture-of-Experts (MoE):** To address the challenges of (1) model scalability and (2) diverse user behavior on X, we employ the sparsely gated Mixture-of-Experts (MoE) (Shazeer et al., 2017) strategy used by BotMoE (Liu et al., 2023). This approach leverages specialized experts conditioned on (learned) user sub-communities, enabling community-aware representation learning.

a) **Expert Assignment and Gating:** For each modality (text, DNA), user embeddings are fed into a **gating network**:

$$G(\mathbf{x}) = \text{softmax}(\text{KeepTopK}(W_g \mathbf{x}, k)) \quad (7)$$

where W_g are learnable weights, $\mathbf{x}$ is the input embedding, and KeepTopK retains the top k gating scores for sparse expert activation.

b) *Expert Aggregation*: The final representation for modality $m \in \{t = \text{text}, d = \text{DNA}\}$ is a weighted sum of expert outputs:

$$\mathbf{z}_i^{(m)} = \sum_{j=1}^n G(\mathbf{x}_i^{(m)})_j E_j(\mathbf{x}_i^{(m)}) \quad (8)$$

where $E_j(\cdot)$ denotes the j^{th} expert (a two-layer MLP), and $G(\mathbf{x}_i^{(m)})_j$ is the gating score for expert j .

c) *Feature Fusion and Consistency*: After expert aggregation, representations across modalities are fused using a multi-head **self-attention** mechanism:

$$\{\tilde{\mathbf{z}}^{(t)}, \tilde{\mathbf{z}}^{(d)}\} = \text{SelfAttn}(\{\mathbf{z}^{(t)}, \mathbf{z}^{(d)}\}) \quad (9)$$

Additionally, we compute a **consistency matrix** between the modalities and downsample it:

$$\mathbf{z}^{(con)} = \text{flatten}(\text{Conv2D}(\text{sample}(\mathbf{M}))) \quad (10)$$

where $\mathbf{M}$ is the attention matrix across modalities, and the convolution extracts cross-modal consistency signals.

d) *Classification Head*: The fused representations and consistency vector are concatenated and passed through a final classifier:

$$y_i = W_o \cdot [\tilde{\mathbf{z}}^{(t)}, \tilde{\mathbf{z}}^{(d)}, \mathbf{z}^{(con)}] + b_o \quad (11)$$

where W_o and b_o are learnable parameters.

e) *Loss Function*: We optimize a **cross-entropy loss** with two regularization terms:

$$\mathcal{L} = - \sum_{i \in \mathcal{Y}} t_i \log y_i + \lambda_1 \sum_{\theta} \|\theta\|_2^2 + \lambda_2 \sum_{i \in \mathcal{Y}} \sum_{m \in \{t, d\}} BL(\mathbf{x}_i^{(m)}) \quad (12)$$

where θ denotes model parameters (used for L2-regularization) and $BL(\cdot)$ is the **balancing loss** from Shazeer et al. (2017):

$$BL(\mathbf{x}) = w_{imp} \cdot CV(G(\mathbf{x}))^2 + w_{ld} \cdot CV(P(\mathbf{x}, i))^2 \quad (13)$$

Here, CV is the coefficient of variation, encouraging balanced expert usage.

f) *Sessions 20-21 - Text, DNA*: For sessions 20 and 21, we employed the MoE architecture with the following modalities:

- Text: User description + five tweets sampled evenly across their timeline (partitioning their posts into equal chunks), both encoded with TwHIN-BERT.
- DNA: content + time, encoded with MobileViT-small.

The architecture (illustrated in Figure 2) can be summarized as follows:

- Each modality employs two to three experts, activated via sparse gating to promote community-aware specialization.
- Outputs from experts are fused via multi-head self-attention, with cross-modal consistency extracted via convolutional filters over attention matrices.
- The fused and consistent representations are passed through a two-layer classification head.

Strengths:

- *Efficiency*: It is the smallest of our multimodal models, requiring less time to train and perform inference.
- *Community-aware specialization*: Each expert learns patterns within specific user sub-communities, improving generalization across diverse bot behaviors, and potentially simplifying the learning process due to reduced per-expert user diversity.
- *Balanced expert utilization*: The balancing loss encourages fair expert usage, preventing mode collapse and overfitting.

Challenges:

- *Hyperparameter tuning*: Despite being a smaller model, the sparsely gated MoE architecture has several hyperparameters (e.g., number of experts, expert hidden dimension, weight initialization for the gates) requiring careful tuning.

The MoE was our strongest model, achieving perfect precision in English, and the highest recall in all of the French sessions. We reason that the model did not require treatment of class imbalance because the reduced per-expert user-diversity made it easier for experts to learn informative representations.

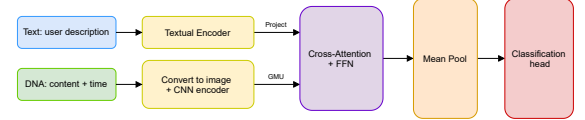


Fig. 1: Text-DNA Cross-Attention bot detection framework.

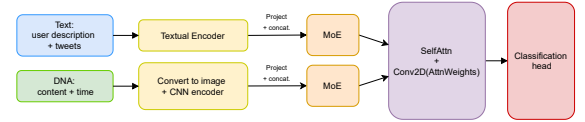


Fig. 2: Text-DNA MoE bot detection framework.

IV. DISCUSSION

Over the course of our study, we have found that key challenges in bot detection on X include manipulated features and diverse user behavior. The former necessitates a model that captures user behavior from a variety of perspectives to increase the likelihood of encountering obvious signals. The latter introduces variance to the training data, and our MoE approach shows that a large model can be successfully replaced with a smaller divide-and-conquer architecture.

An interesting aspect of the MoE is that the expert specializations emerge in a black box fashion. We reason that the classification task is broken down into two steps more explicitly than in other architectures:

- (Expert gating) What type of user is this instance?
- (Expert) Among those types of users, what makes this one stand out?

Session	Model	Total Users	Acc.	Prec.	Rec.	F1
11	RF	420	0.9929	0.0000	0.0000	0.0000
12	RF	393	0.9669	0.7333	0.5500	0.6286
13	RF	394	0.9949	1.0000	0.9048	0.9500
14	RF	402	0.9577	0.7143	0.6897	0.7018
15	N/A	—	—	—	—	—
16	CA	438	0.8151	0.1429	0.5714	0.2286
17	CA	306	0.8105	0.1970	0.7222	0.3095
18	CA	455	0.9275	0.5714	0.5263	0.5479
19	CA	288	0.9132	0.6176	0.6364	0.6269
20	MoE	448	0.9777	1.0000	0.6774	0.8077
21	MoE	283	0.9682	0.8800	0.7857	0.8302

TABLE I: Performance of our bot detectors across sessions. Precision, recall, and F1 are computed for the bot class.

V. CONCLUSION AND FUTURE WORK

blah blah...

REFERENCES

- John Edison Arevalo, Thamar Solorio, Manuel Montes-y G’omez, and Fabio A. Gonz’alez. Gated multimodal units for information fusion. *CoRR*, abs/1702.01992, 2017. URL <https://arxiv.org/abs/1702.01992>.
- David M. Blei, Andrew Y. Ng, and Michael I. Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.
- Clayton A. Davis, Onur Varol, Emilio Ferrara, Alessandro Flammini, and Filippo Menczer. Botornot: A system to evaluate social bots. In *Proceedings of the 25th International Conference Companion on World Wide Web*, pages 273–274. ACM, 2016.
- Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022. URL <https://arxiv.org/abs/2203.05794>.
- Loukas Ilias, Ioannis Michail Kazelidis, and Dimitris Th. Askounis. Multimodal detection of bots on x (twitter) using transformers. *arXiv preprint arXiv:2308.14484*, 2023. URL <https://arxiv.org/abs/2308.14484>.
- Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988. IEEE, 2017. doi: 10.1109/ICCV.2017.324. URL <https://arxiv.org/abs/1708.02002>.
- Yuhan Liu, Zhaoxuan Tan, Heng Wang, Shangbin Feng, Qinghua Zheng, and Minnan Luo. Botmoe: Twitter bot detection with community-aware mixtures of modal-specific experts. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR ’23*, pages 485–495, Taipei, Taiwan, July 2023. ACM. doi: 10.1145/3539618.3591646.
- Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software*, 2(11):205, 2017. doi: 10.21105/joss.00205.
- Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. <https://www.sbert.net>, 2021. Accessed: 2025-04-25.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *CoRR*, abs/1701.06538, 2017. URL <http://arxiv.org/abs/1701.06538>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008. Curran Associates, Inc., 2017. URL https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *arXiv preprint arXiv:2002.04745*, 2020. URL <https://arxiv.org/abs/2002.04745>.